

SANE Manipulation: A Sensor-Anchored Execution Harness for LLM/VLM Robot Planning

Anonymous Author(s)

Abstract—Large language and vision-language models (LLMs and VLMs) can plan robot manipulation zero-shot, leveraging pretraining corpora far broader than typical robot-learning demonstrations. Their modular, gray-box nature also allows plans to be inspected before execution, unlike end-to-end vision-language-action (VLA) policies. However, a decoupled Sense-Plan-Act pipeline still requires the model to specify physical quantities such as release height, grasp width, or target position, which pretraining does not ground in the scene. We present SANE (Sensor-Anchored Numeric Execution), an execution harness inserted between the high-level planner and low-level skill executor. While the planner handles symbolic decisions, SANE recomputes action parameters from sensor measurements or committed actions. We motivate SANE through a structured study of LLM/VLM pick-and-place on a real UR5cb, identifying six recurring failure modes that persist even under perfect perception and therefore stem from physical reasoning rather than perception. SANE resolves five of the six modes for common manipulation tasks. In a 120-trial real-hardware benchmark using a unified model across both grounding modalities, the LLM implementation achieves 98.3

I. INTRODUCTION

A vision-language-action (VLA) model maps perception and instruction directly to low-level control, in one policy trained end-to-end. The architecture is elegant, but it inherits the oldest bottleneck of imitation learning: the policy only knows what its demonstration data covers. Moreover, paired vision-action trajectories are expensive to collect at the scale a general-purpose manipulator needs. An LLM or VLM, by contrast, is pretrained on a broader and cheaper corpus: text, code, and image-text pairs at internet scale. That corpus already encodes much of what manipulation requires, including object affordances, spatial and arithmetic reasoning, and precondition ordering. None of this is robot-specific, and all of it transfers through code rather than through a demonstration dataset [?], [?], [?], [?]. A decoupled Sense-Plan-Act pipeline makes this architectural bet: the model plans in language, and a separate execution layer turns the plan into motion.

A decoupled pipeline still has to ask the model, somewhere, to commit to numbers: a release height, a grasp width, a target position. Nothing in an LLM or VLM’s pretraining grounds those numbers in the physical scene the robot acts in. In simulated validation this cost is easy to miss: an object’s height or the distance a gripper must travel are never measured there, but asserted, correctly, by construction [?], [?], [?]. In the real world, by contrast, every such quantity has to be computed: from noisy depth, an imperfect camera, or a hand-authored scene file that may not match the physical table it describes.

This paper closes that gap with an execution harness. We proceeded in three steps. First, we built an experimental baseline: a ROS2 Sense-Plan-Act stack in which an LLM or VLM decomposes a natural-language command into a plan of parameterized skills, using any of six established reasoning strategies [?]. The baseline supports two grounding regimes: an LLM planner with perfect perception, reading an exact scene description, and a VLM planner perceiving the scene through a real camera. Second, we ran a structured investigation of the baseline on pick-and-place tasks with a real UR5cb manipulator. It surfaced six recurring failure modes, each traced from a hardware failure back to a specific model output. Crucially, the failures also occur in the LLM implementation, whose perception is perfect by design. The root cause is therefore how the model reasons about physical quantities, not how the scene is perceived. Third, guided by this analysis, we extended the architecture with a third layer between the high-level planner and the low-level executor: SANE (Sensor-Anchored Numeric Execution). The planner keeps the symbolic decisions, which object to move and where; SANE recomputes each action’s physical quantities from a sensor measurement or from an action already committed in the plan. The result is a complete robotic pipeline that couples high-level LLM/VLM planning, a deterministic execution harness, and low-level control primitives. By applying SANE we resolve five of the six failure modes, and a 120-trial real-hardware benchmark validates the design.

The contributions of this paper are threefold:

- A structured investigation of how LLM/VLM planners, complemented with a set of predefined low-level control primitives, fail on real pick-and-place, yielding a taxonomy of six recurring failure modes that arise independently of perception quality (Section IV).
- SANE, a deterministic execution harness for decoupled LLM/VLM manipulation planning, presented as an explicit third architectural layer between planner and executor. This is the paper’s main contribution (Section V).
- A 120-trial real-hardware validation of the harness, using a single unified model for both grounding modalities. It shows that the one failure SANE does not yet resolve, multi-object placement collision, dominates the residual failures of both implementations (Section VI).

The rest of the paper is organized as follows. The related work (Section II) argues why a decoupled Sense-Plan-Act architecture with LLM/VLM planners is preferable to an end-to-end VLA, on data-efficiency and diagnosability grounds,

and reviews the simulation-based benchmarks this work builds on. The experimental baseline (Section III) describes the two-layer system we built, its two perception regimes, and the exact point where the harness will later be inserted. The failure investigation (Section IV) reports the six failure modes and the evidence that they are reasoning failures rather than perception failures. Section V introduces SANE and shows how it resolves five of the six. The evaluation (Section VI) validates the harness with a 120-trial benchmark on real hardware. Section VII discusses limitations, and Section VIII concludes.

II. RELATED WORK

From symbolic planning to LLM/VLM planners. Robotic planning has long been structured around a Sense-Plan-Act pipeline [?], first realized through hierarchical and symbolic planners that search a discrete action space for a provably goal-reaching sequence [?]. These planners give strong guarantees but require a hand-specified world model. Foundation models relaxed that requirement. They ground natural-language instructions in robotic affordances [?], act as zero-shot planners [?], and generate motion plans directly from instructions [?]. On top of these capabilities, a range of reasoning strategies has been proposed: reactive single-step prompting, interleaved reasoning and acting (ReAct) [?], iterative self-critique [?], [?], tree search [?], and chain-of-thought with self-consistency voting [?], [?]. Recent surveys catalogue this space [?], [?], [?], [?], [?], [?]. Our contribution is orthogonal: we propose no new reasoning strategy, and our evaluation holds the strategy fixed. We address a failure axis that sits underneath all of them: how a chosen action’s numeric parameters are computed. That axis persists whatever strategy produced the action.

Why Sense-Plan-Act with LLM/VLM planners over a VLA. An alternative architecture collapses perception, reasoning, and control into one vision-language-action policy [?], [?], [?], [?]. We argue for the decoupled architecture on two grounds. The first is data. A VLA must be trained or fine-tuned on paired vision-action data, which is scarce, embodiment-specific, and costly to collect. A decoupled pipeline instead plans in text, or text conditioned on an image, and emits a sequence of parameterized skills. The model’s pretrained knowledge, spatial reasoning, arithmetic, common-sense physics, and ordering logic, becomes available zero-shot, through code, without a single task-specific demonstration [?], [?], [?], [?]. The second ground is diagnosability. When an end-to-end policy fails, it is hard to tell a misperception from a reasoning error or a motor-control error [?]. A decoupled pipeline exposes its intermediate plan, so every failure can be traced to a specific stage and a specific model output. The failure analysis of this paper (Section IV) would not have been possible in an end-to-end system; only its symptoms would have been visible. The cost of decoupling is that the model must produce numeric parameters it was never trained to get right. **The harness proposed in this paper addresses precisely this weakness.**

Simulation, sim-to-real, and the closest prior work. The sim-to-real gap is a long-studied concern in robot learning, primarily for learned low-level control [?]. Physics simulators such as Gazebo [?], MuJoCo [?], and CoppeliaSim [?] narrow that gap by modeling contact dynamics and actuation limits [?]. The gap we document is different in kind. It concerns whether a *model-computed* geometric quantity is trustworthy at all, a question a symbolic benchmark never poses, because it never confronts the model’s number with a contradicting sensor. The closest prior work is Favali et al. [?], which introduces a task-complexity taxonomy and the TS/TSR/AETS metrics for benchmarking LLM-based robotic planning across six reasoning strategies, evaluated in a symbolic simulator with exact ground truth. We adopt the same evaluation philosophy and the same metric family, but on real hardware, where ground truth for safety and success is not automatically available.

III. THE EXPERIMENTAL BASELINE

This section describes our experimental baseline: a two-layer Sense-Plan-Act stack, a high-level planner over a low-level skill executor, with no correction layer in between. The baseline is what we investigate in Section IV; understanding what it trusts, and from whom, is what makes the harness’s role clear.

A. Architecture

The baseline is a ROS2 stack organized as a Sense-Plan-Act pipeline (Figure 1). A task interface, a terminal client or a web GUI, forwards a natural-language command to a planner node. The planner decomposes the command into a plan: an ordered sequence drawn from a small, fixed set of parameterized skills, `approach`, `pick`, `release`, `move_home`, and `wait`. A plan-manager node validates and sequences the plan, then drives a skill-executor node **that commands** the real UR5cb driver. Every planning call supports six interchangeable reasoning strategies, from a single-step reactive baseline to self-consistency voting and tree search, selectable without touching the pipeline itself.

B. Two implementations, two perception regimes

The baseline supports two planner **implementations** that differ only in how the scene enters the plan.

LLM implementation: perfect perception. The static-scene LLM planner reads a hand-authored scene description that lists every object’s identity, position, and size exactly. Perception is perfect by construction: there is no camera, no noise, and no detection error. Whatever goes wrong in this **implementation** cannot be blamed on perception.

VLM implementation: real perception. The camera-grounded VLM planner perceives the scene through a calibrated RGB-D camera. Object positions and sizes come from detection and depth deprojection, so they carry real sensor noise, occlusion, and parallax error.

Both **implementations** plan over the same physical rig, **shown in Figure 2**: a UR5cb manipulator with an OnRobot RG2 parallel gripper and a calibrated RGB-D camera, acting

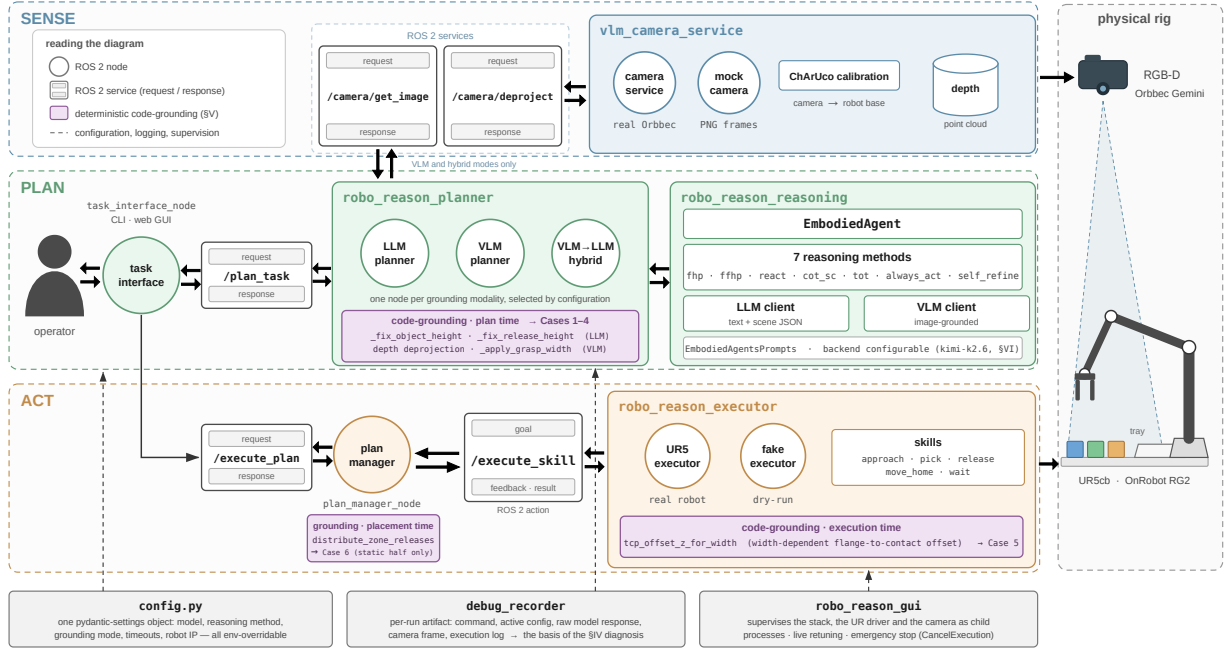


Fig. 1. The experimental baseline and the SANE harness. A high-level LLM/VLM planner decomposes a natural-language command into a plan of parameterized skills; a low-level executor drives the UR5cb. SANE (Section V) sits between the two and overrides the plan’s numeric geometry before execution.

on a tabletop scene of four color-distinguished cubes and a tray. The tray is registered in the scene as a target zone: a placement region with real spatial extent, which several tasks use as their destination. The task families are pick-and-place, sorting and stacking, and arithmetic value-mapping, in which the object to move must be computed from a stated numeric rule rather than named. The evaluation (Section VI) details the full protocol.

C. Instrumentation

Two design choices make the pipeline reproducible and easy to instrument. First, every runtime parameter, model choice, reasoning strategy, grounding modality, timeouts, robot address, is a field on one centralized settings object, overridable by environment variable with no code change. Experimental conditions are therefore configuration, not code branches. Second, every planning call records a per-run debug artifact: the command, the active configuration, the raw model response, the execution log, and, in VLM modes, the camera frame the plan was grounded in. This is what made the failure investigation possible: each failure in Section IV was traced from a hardware event back to a specific field in a specific logged response, not inferred after the fact.

One property of the baseline deserves emphasis, because the rest of the paper turns on it. As described here, the baseline trusts every numeric field of a generated plan. If the model says an object is 9 cm tall, the executor lifts 9 cm. The next section shows what that trust costs on real hardware; Section V then inserts SANE between plan validation and skill execution, exactly at the point where those numbers stop being suggestions and become motion.

IV. A STRUCTURED INVESTIGATION OF MANIPULATION PLANNING FAILURES

To understand where LLM/VLM planning actually breaks on a real robot, we ran a structured investigation of the baseline on pick-and-place. We exercised both planner implementations across all three task families, and traced every hardware failure back to its root cause using the baseline’s per-run debug artifacts: the raw model response, the active configuration, and, in VLM mode, the captured camera frame. The investigation yielded six recurring failure modes, summarized in Table I. They fall into three families:

- **Measurement failures:** the plan uses a quantity only a sensor can know, but takes the model’s guess instead (double-counted stacking height, silhouette-edge width sampling).
- **Embodiment failures:** the plan’s abstraction of the robot or the scene is simpler than the real thing, a point where the target has extent, a constant where the gripper’s geometry varies (phantom top-surface grasp, point-distance target matching, width-dependent tool offset).
- **Contact-dynamics failures:** the plan constrains final positions but not what bodies do over time (sequential multi-object placement collision, still unresolved).

One finding shapes everything that follows. These failures do not depend on perception quality. The phantom top-surface grasp appeared first in the VLM implementation, was fixed there, and then recurred independently in the LLM implementation, whose scene description is exact by construction. The benchmark of Section VI later confirmed

the same pattern for the placement collision: it dominates *both* **implementations**. The problem is how the model reasons about physical quantities when turning a decision into numbers, not how the scene was perceived. This is why the harness of Section V operates at the execution level, correcting the numbers a plan carries, and not at the perception level.

Below we develop the two most instructive resolved cases in full, then treat the still-unresolved collision failure on its own.

A. Case study 1: phantom top-surface grasp

The executor deliberately grasps objects mid-body rather than at the top surface. The pick descent is $d = \min(0.5h, d_{\max})$, where h is the object’s height and $d_{\max}$ is a clamp that keeps the gripper’s rigid mounting body away from tall objects. A mid-body grasp is simply more stable than pinching a top edge.

The paired release action, however, computes the lift height from the object’s *full* height. That computation silently assumes a top-surface grasp. The assumption is wrong by construction whenever the pick descended mid-body, which is always. The result is a systematic overshoot of roughly $0.5h$: objects are dropped from a height proportional to half their own size, rather than from fractions of a centimeter.

We first fixed this in the VLM **implementation**, by deriving the object’s height from measured depth instead of the model’s reported size. **The same failure then recurred, independently, in the LLM implementation**: its prompt still asked the model to compute the lift from the scene’s declared object height, the exact assumption the earlier fix had already identified as wrong. Fixing the failure once, in one **implementation**, did not fix it everywhere it existed. The error is not a mistake one prompt makes. It is a mismatch between what the model is asked to assume and what the gripper actually does, and it therefore needs an architectural fix, not a prompt fix.

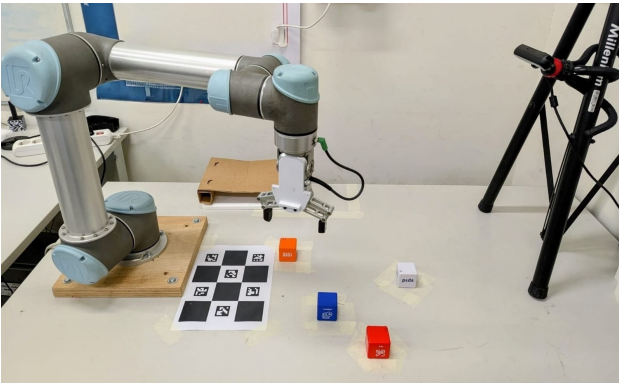


Fig. 2. The physical rig: the UR5cb manipulator with the OnRobot RG2 parallel gripper, the RGB-D camera on its tripod (right), the calibration board used for camera-robot registration, and the color-distinguished cubes on the workspace table.

B. Case study 2: double-counted stacking height

The scene convention represents each object by a base reference height and a size, so base plus height reconstructs the top surface. Every stacking prompt relies on this formula. **In the camera-grounded pipeline, the scene-grounding step deprojects the target’s pixel location into a depth-measured world coordinate. An early version wrote that measured value, already the true top surface, into the scene description as the target’s base reference, while leaving the model’s separate, unverified height estimate untouched. The planner then applied its normal stacking formula and added the guessed height on top of an already-correct value.** Every stack-on-target release was inflated by however wrong that guess happened to be.

The lesson generalizes Case study 1. A sensor-measured quantity must be inserted *into the representation the planner already expects*, consistently with its conventions, rather than mixed with model guesses that the planner will keep combining by formula.

C. The **unresolved** case: sequential-placement collision

Unlike the five other failures, this one is not **resolved** by the harness. When a task places several objects into the same zone in sequence, the harness spaces the *final* release positions so they cannot overlap. That is only the static half of the problem. Nothing constrains the arm’s trajectory while it approaches the n -th release past the $(n - 1)$ objects already in the zone. Nothing constrains a released object’s residual motion from displacing a neighbor afterward, either. Both halves are three-dimensional, time-dependent contact problems. They cannot be solved by correcting a single scalar, which is exactly what makes them different in kind from Case studies 1 and 2.

We diagnosed this failure from the same debug artifacts as the others, but by the time it was isolated it had already become the dominant failure mode of the benchmark, in both **implementations**. We report it as an identified, partially mitigated, **unresolved** problem; Section VII returns to what **resolving** it would require.

V. RESOLVING THE FAILURES WITH SANE

The failures of Table I share one root cause: the plan relies on the model to compute physical quantities that could instead be obtained exactly, either from a sensor measurement or from an action the plan has already committed to. This observation motivates the paper’s main contribution. We extend the pipeline from two layers to three by inserting SANE (Sensor-Anchored Numeric Execution) between the high-level planner and the low-level executor. SANE is a deterministic layer: it receives each validated plan before execution and replaces the numeric parameters most exposed to the identified failure modes with values it computes itself, using only information available at plan time.

The design principle fits in one sentence: *the model decides what to do and where; SANE computes how much*. The model still chooses which object to pick and which zone to place it in. Geometric arithmetic is pulled out of

TABLE I

THE SIX FAILURE MODES. “PHYSICAL PROPERTY REQUIRED” MARKS WHAT MAKES EACH FAILURE IMPOSSIBLE TO OBSERVE IN A SYMBOLIC OR SCRIPTED SIMULATOR.

Failure	Symptom and root cause	Physical property required
Phantom top-surface grasp	The release lift is set to the object’s full height, assuming the object was grasped at its top. The executor actually grasps mid-body, descending by $d = \min(0.5h, d_{\max})$. Net effect: a systematic release overshoot of roughly half the object’s height. Recurred independently in both implementations .	Real gripper kinematics: mid-body grasping is a deliberate stability choice, not a simulator default.
Double-counted stacking height	During camera-based scene grounding, the target’s top surface was already measured correctly from depth. The planner’s stacking rule then added the model’s separate, unverified height estimate on top of it a second time.	A real depth measurement of an already-correct surface height.
Point-distance target matching	A placement genuinely inside the tray’s real footprint, but far from its registered center, was scored as a miss by a point-distance check.	A real target with spatial extent of tens of centimeters, not a point.
Silhouette-edge width sampling	Grasp width was read from depth exactly on the object’s silhouette boundary, the single worst location for valid depth due to occlusion and parallax. One failed edge sample aborted an otherwise valid plan.	Real depth noise at object boundaries; absent for a rendered or symbolic bounding box.
Width-dependent tool offset	The gripper’s fingers pivot, so the flange-to-contact distance is a function of the commanded opening width, not a constant. A single calibrated offset is wrong for any object of a different width.	Real finger kinematics, absent from a symbolic action space.
Sequential-placement collision (unresolved)	Zone-aware spacing keeps a zone’s <i>final</i> release positions from overlapping. It does not stop the arm clipping an already-placed object en route to the next release, nor a released object settling into a neighbor. Dominant residual failure in the 120-trial benchmark: all 22 non-empty operator notes describe a collision or a fallen object during a multi-object task, in both implementations .	Real multi-body contact between placed objects and the approach trajectory, over time.

the model and into code wherever the answer is computable from a sensor measurement or from a paired action already in the plan. The harness is implemented as a small set of deterministic correction functions; each is described below, together with the failure it **resolves**.

A. Heights anchored to committed actions and measured depth

Two corrections govern vertical geometry. The first **resolves** the phantom top-surface grasp. For every release, SANE recomputes the lift height from the *same* descent the executor actually commanded during the paired pick, $h - d$ in the notation of Case study 1. A value derived from the plan’s own committed action cannot disagree with what the arm did; a separately model-computed value can, and did.

The second **resolves** the double-counted stacking height. For a stacking target, **SANE measures the target’s real height directly: it takes the depth-measured top surface and the table plane, and uses the vertical distance between the two as the height. The height is never taken as negative, and no minimum is imposed, so a flat zone marked on the table correctly reads as having near-zero height.** It then rewrites the target’s base reference so that the planner’s usual reconstruction of the top surface becomes correct. The measured value is inserted into the representation the planner already expects, instead of being mixed with the model’s own guess.

B. Footprint-based target matching

To decide whether a planned placement position falls within a registered target zone, SANE checks whether the position lies inside the zone’s real footprint, the rectangle spanned by the zone’s size around its center, rather than

measuring the distance to the zone’s center point. This resolves the point-distance matching failure: the tray introduced in the experimental baseline (Section III), a target zone tens of centimeters wide, accepts any placement inside its bounds. A **placement** that exactly echoes a target’s registered coordinates is treated as intentional and left untouched. One that merely drifted into a zone’s footprint is nudged back out along the axis needing the smaller correction. The nudge only fires when the zone’s surface sits meaningfully above the bare table; otherwise the near-table-height generic “table” zone, which legitimately contains most of the workspace, would push objects off the table’s edge.

C. Width-aware tool offset

Because the gripper’s fingers pivot, the flange-to-contact distance depends on the commanded opening width. SANE measures this distance at three representative widths: fully closed, mid-open, and fully open. At plan time, it uses piecewise-linear interpolation between these measurements and sets the executor’s tool transform for each pick according to the planned grasp width. Pick descent is additionally capped so that tall objects are gripped nearer their top, keeping the gripper’s rigid mounting body clear. This **resolves** the width-dependent tool offset failure.

D. Best-effort grasp-width estimation

In the VLM **implementation**, grasp width is estimated by deprojecting the left and right edges of a detected bounding box. Sampling exactly on the silhouette boundary is the worst location for valid depth, and one failed sample used to abort the whole plan, over a parameter that does not even affect whether pick and release are safe. SANE samples slightly inset from the silhouette edge, and isolates width estimation

into its own best-effort call that degrades to the model’s estimate on failure instead of aborting. This **resolves** the silhouette-edge sampling failure, without yet making width depth-verified in every case, a limitation Section VII returns to.

E. Zone-aware spacing: a partial mitigation

For a zone that receives multiple **placements** in one plan, SANE lays the occupants out on a grid centered on the zone’s anchor and clamped to its registered bounds, falling back to an unbounded grid only when no known zone matches. This **addresses** the *static* half of the sequential-placement collision: final positions no longer overlap, nor overflow a zone smaller than the model’s unclamped guess. The *dynamic* half remains **unresolved**. The approach trajectory is not checked against already-placed objects, and a released object’s residual motion is not constrained at all. The evaluation confirms this as the dominant residual failure.

F. Validation of the corrections

Each correction was developed against real, previously failing production runs, not synthetic cases. The baseline’s per-run debug artifacts let us replay the exact inputs that produced a given hardware failure and confirm that the corrected pipeline now produces the hand-derived expected geometry for the same input. This retrospective replay, together with a unit-level regression suite (45/45 passing across all reasoning-strategy implementations at the time of writing), gives us confidence that each correction addresses its diagnosed root cause. What remains is to measure the harness’s effect end to end, which is the subject of the next section.

VI. EVALUATING SANE ON REAL HARDWARE

This section validates the harness. With SANE in place, we ran a 120-trial real-hardware study comparing the two **implementations** of the baseline: the static-scene LLM planner and the camera-grounded VLM planner. The study answers two questions. First, how reliable is the full three-layer system on real hardware? Second, what fails once the five **corrections** are active, and does it match the one failure SANE does not **resolve**?

A. Design

The study is anchored to the task and environment complexity taxonomy and the TS/TSR/AETS metrics of Favali et al. [?], adapted to a physical rig. Six task conditions cross three difficulty axes, task length, reference specificity, and affordance and perception load, each at an easy and a hard level, over the real scene of four color-distinguished cubes and a tray: `pp_easy/pp_hard` (pick-and-place, one vs. four objects), `sort_easy/sort_hard` (stacking and sorting), and `arith_easy/arith_hard` (the object to move must be computed from a stated numeric mapping rather than named, isolating reasoning load from perception load). Each condition ran 10 times per **implementation**, 120 trials in

TABLE II
OVERALL RESULTS, 60 TRIALS PER **PLANNER IMPLEMENTATION** (120 TOTAL), WITH SANE IN PLACE.

Planner	n	TS	TSR	AETS
LLM (kimi-k2.6)	60	98.3%	98.3%	0.1269
VLM (kimi-k2.6)	60	85.0%	89.2%	0.1149

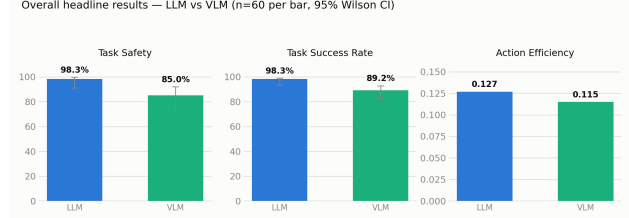


Fig. 3. Overall TS/TSR/AETS summary, LLM vs. VLM **implementation**, with SANE in place.

total. A single reasoning strategy, chain-of-thought with self-consistency, was held fixed throughout: the most safety-robust strategy in [?], chosen so that strategy variance is not stacked on top of the comparison under test. Both **implementations** use the same underlying model (kimi-k2.6), text-only and image-grounded respectively, so grounding modality is the only variable between them. The accompanying video presents the physical rig and representative trials from all six conditions.

B. Metrics, adapted for physical ground truth

TS (task safety), TSR (task success rate), and AETS (action efficiency on task success) are fully automatic in a symbolic simulator, where damage and completion are exactly derivable from simulator state. Our UR5cb has no collision or force-fault sensing in its control path, and no vision-based verification of the final scene. We therefore adapt the same three metrics to a semi-automatic protocol. Everything the system can know automatically is logged per trial by the baseline’s debug instrumentation. Immediately after each trial, the operator supplies exactly two numbers, whether the trial was safe and how many sub-tasks completed, from which TS, TSR, and AETS are computed identically to their original definitions.

At $n = 10$ per cell, point estimates alone invite justified pushback, so every reported TS and TSR carries a 95% Wilson score interval (TS is an exact per-trial proportion; TSR pools successes over sub-tasks required $\times n$). One comparison was declared before any data was collected: a two-sided Fisher’s exact test, LLM vs. VLM, on `arith_hard` full-trial success, at $\alpha = 0.05$. It is binarized per trial rather than pooled across sub-tasks, since sub-tasks are not independent: one wrong decision tends to misplace every object in a trial together. Everything else is reported descriptively.

C. Results

Table II reports the headline result. Both **implementations** share the same harness and the same underlying model, so

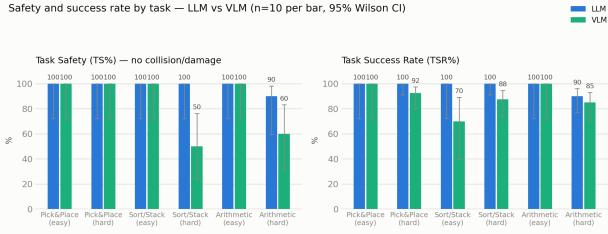


Fig. 4. Task safety and task success rate by task condition, with 95% Wilson intervals.

the residual gap between them is neither a grasp-geometry gap nor a model-capability gap; it is attributable to grounding modality. By task condition (Figure 4), the gap concentrates on the two hardest multi-object conditions, and predominantly in safety. TS is 100% for both **implementations** on every easy condition and on `pp_hard` and `sort_hard` for the LLM **implementation**, but drops to 50% (VLM, `sort_hard`) and 60%/90% (VLM/LLM, `arith_hard`), the conditions where the most objects share a zone at once. Neither **implementation** shows the sharp success-rate collapse an earlier pilot on this task matrix, run with different per-modality models, had found on the arithmetic conditions (30%/25% VLM TSR against 100%/97% LLM). With a single unified model, `arith_easy` is 100%/100% and `arith_hard` is 90%/85% (TSR, LLM/VLM): a real but modest gap, not a collapse. The one pre-registered test, Fisher’s exact on `arith_hard` full-trial success, shows no detectable difference at this sample size (LLM 6/10, VLM 4/10, $p = 0.66$), consistent with a modest gap that $n = 10$ per cell cannot resolve to significance.

D. What fails with the harness in place

Reading all 22 non-empty operator failure notes from the 120 trials, rather than trusting aggregates alone, was the key analysis step. Every note describes either a collision between objects (10) or an execution or grasp failure, a slipped grasp, a release too high, a dropped object (12), during a multi-object task. Zero notes describe a wrong object chosen for reasoning or value-mapping reasons, and zero an incomplete enumeration (Figure 5). The pattern holds in both **implementations**: the LLM **implementation** contributes 4 of the 22 notes, all on `arith_hard`, and the VLM **implementation** the remaining 18, spread across the three multi-object conditions roughly in proportion to trial counts.

This is the validation the section set out to obtain. With the five corrections active, the dominant residual failure is precisely the one SANE does not yet **resolve**, the sequential-placement collision, and it dominates **both implementations**, regardless of which planner grounded the scene. Zero notes implicate the model’s choice of object or its task decomposition. The remaining grasp and execution notes describe physical instability during multi-object trials rather than the **resolved** height and matching classes. The residual failure is a property of the **unresolved** contact-dynamics problem, not of a perception modality.

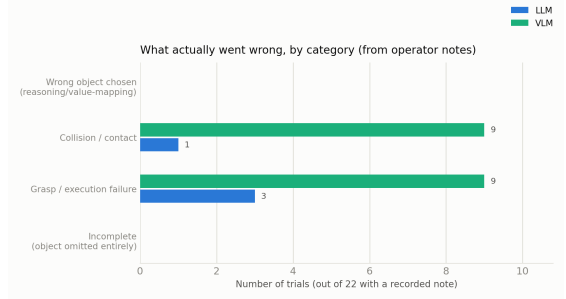


Fig. 5. Failure-mode breakdown, derived from the 22 non-empty operator notes out of 120 trials.

VII. DISCUSSION AND LIMITATIONS

Implication for evaluation practice. Symbolic and scripted simulation is necessary but not sufficient for validating an LLM/VLM robot planner. It validates reasoning strategy and task decomposition. By construction it cannot exercise the failure modes of Section IV, because it never asks the model to compute a quantity a physical sensor could contradict. We therefore propose a *geometry-grounding audit* as a standard step before hardware deployment: identify every numeric quantity a prompt asks the model to compute itself, and ask whether a sensor or a paired action already determines it more reliably. Each quantity that fails the audit is a candidate for a SANE-style correction.

The cost of the architectural choice. The related work argued for a decoupled Sense-Plan-Act architecture over an end-to-end VLA on data-efficiency and diagnosability grounds. The failure investigation is the other half of that trade. **The decoupled architecture asks the model for physical quantities it was never trained to estimate, and the six documented failure modes are the consequence; the harness is the extra layer this architecture needs to be reliable in practice.** We read this as a cost worth paying, not a case against the architecture. **Every resolved failure was fixable once diagnosed**, precisely because the pipeline stayed decoupled enough to diagnose it.

The unresolved collision case. The sequential-placement collision is a trajectory-level, multi-body contact problem, and the static spacing mitigation of Section V does not address its dynamic half. The most natural extension is an obstacle-aware approach skill that treats already-placed objects as static obstacles, optionally paired with a settle-time check after each placement. **This is the next addition we plan.**

Generalizability. Our evidence comes from one manipulator (UR5cb), one gripper, one camera rig, and a single underlying model (`kimi-k2.6`) used for both **implementations**. We do not claim that the six specific failures of Table I recur verbatim on different hardware. We claim that the structural pattern is a property of the architecture, not of this robot: geometry computed by a model instead of measured, and multiple real bodies sharing space over time. Grasp width is the one quantity we still cannot cross-check against depth in every case, since a single top-down sample determines height

unambiguously but not left-right extent; the best-effort inset sampling of Section V reduces this gap without **eliminating** it.

VIII. CONCLUSION

We presented SANE (Sensor-Anchored Numeric Execution), an execution harness that turns a two-layer LLM/VLM Sense-Plan-Act pipeline into a three-layer one. **The planner keeps the symbolic decisions; SANE recomputes each action’s physical quantities from sensor measurements or from actions already committed in the plan; the executor turns the corrected plan into motion.** The harness is motivated by a structured investigation of pick-and-place on a real UR5cb manipulator, which surfaced six recurring failure modes, invisible to symbolic simulation by construction. The failures arise even under perfect perception, so the problem lies in reasoning about physical quantities, not in perceiving the scene, and the harness accordingly operates at the execution level. **By applying SANE, we resolve five of the six failure modes.** A 120-trial real-hardware benchmark, using a single unified model for both grounding modalities, validates the harness and shows the sixth failure, a multi-object placement collision, as the dominant residual failure in both **implementations**. **Resolving** that failure’s dynamic, trajectory-level half as a new skill, and a depth-verified treatment of grasp width, are the most direct next steps. More broadly, we propose the geometry-grounding audit as a standard companion to task-complexity benchmarking for any LLM/VLM planner headed to physical hardware.